الجامعة الدولية للرباط
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⴳⵔⴰⵖⵍⴰⵏⵜ ⵏ ⵕⵕⴱⴰⵟ
Université Internationale de Rabat

Web Avancé et Développement mobile

PROF. RAHHAL Ibrahim
EMAIL: BRAHIM.RAHHAL@UIR.AC.MA

Année Universitaire: 2025-2026

Développement Mobile sous Android

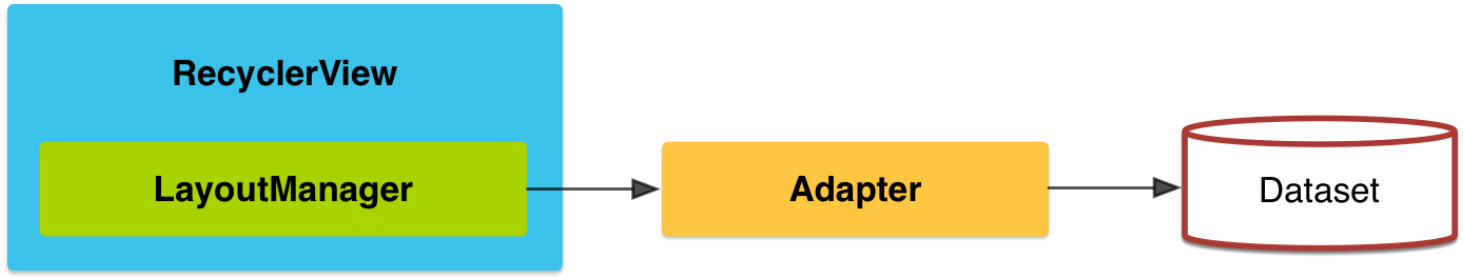
Gestion des listes

Introduction

Objectif : programmer une application pour afficher et éditer une liste d'items:

- La liste est stockée dans un tableau type ArrayList ou dans une Base de Données Realm.
- L'écran est occupé par un RecyclerView (on utilisait Anciennement des ListView) une vue spécialisée dans l'affichage de listes quelconques.

L'intermédiaire entre les données et la vue est **un adaptateur**: un objet qui permet d'afficher un item dans le RecyclerView.



Recette pour les Liste

Étape A — Data (le modèle)

Une classe Etudiant + une ArrayList<Etudiant>

Étape B — Item UI

Un layout d'une ligne (ex: etudiant.xml)

Étape C — Adapter (avec ViewHolder)

3 méthodes seulement à retenir:

- getItemCount() : combien d'items
- onCreateViewHolder() : créer la "carte" (inflate)
- onBindViewHolder() : remplir la carte avec les données

Étape D — Brancher dans l'activité

Dans onCreate() :

- recycler.setLayoutManager(new LinearLayoutManager(this))
- recycler.setAdapter(adapter) (+ optionnel setHasFixedSize(true) si tailles identiques)

Utilisation des Items : Tableau constant Java

Etape 1: la représentation des données : créer une classe pour représenter les items et rajouter le constructeur, tous getters et setters et la méthode toString.

```
public class Etudiant {
        public String nom_prenom; // nom de l'étudiant
        public int code_etud; // code de l'étudiant
        Etudiant(String nom, int code) {
            this.nom_prenom = nom;
            this.code_etud = code;
        }
```

Etape 2: insérer les données initiales à l'aide d'un tableau constant Java (final signifie constant, initdata ne pourra pas être réaffecté).

```
final Etudiant[] initdata = {
        new Etudiant( nom: "Etudiant1",  code: 1223),
        new Etudiant( nom: "Etudiant2",  code: 1527),
        new Etudiant( nom: "Etudiant3",  code: 1657),

};
```

Utilisation des Items : Tableau constant Java

L'étape 3: consiste à recopier les valeurs initiales dans un tableau dynamique de type ArrayList <Etudiant > :

```
private ArrayList<Etudiant> liste;
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    // copie du tableau initdata dans le ArrayList
      liste = new ArrayList<>(Arrays.asList(initdata));

}
```

le container ArrayList<type> : un type de données générique, paramétré par le type des éléments mis entre <. . .>.

```
//le type entre <> à droite est facultatif.
ArrayList<TYPE> liste = new ArrayList<>();
```

Quelques méthodes utiles pour la gestion des ArrayLists :

- Suppression des éléments: liste.remove(elem ou indice), liste.clear.
- Ajout des éléments : liste.add(elem)
- Récupération des éléments : liste.size(), get(indice), contains(elem), indexOf(elem) .

Utilisation des Items : Un tableau dans les ressources

Etape 1: la représentation des données : créer une classe pour représenter les items et rajouter le constructeur, tous getters et setters et la méthode toString.

```java
public class Etudiant {
        public String nom_prenom; // nom de l'étudiant
        public int code_etud; // code de l'étudiant
        Etudiant(String nom, int code) {
            this.nom_prenom = nom;
            this.code_etud = code;
        }
```

Etape 2: insérer les données initiales à l'aide d' un tableau dans les ressources dans le fichier res/values/arrays.xml

```xml
<resources>
    <string-array name="nom_prenom">
        <item>Etudiant1</item>
        <item>Etudiant2</item>

    </string-array>
    <integer-array name="code">
        <item>1223</item>
        <item>1527</item>

    </integer-array>
</resources>
```

Utilisation des Items : Un tableau dans les ressources

L'étape 3: Récupérer les ressources tableaux pour remplir le ArrayList :

```
// accès aux ressources
Resources res = getResources();
final String[] noms = res.getStringArray(R.array.nom_prenom);
final int[] code = res.getIntArray(R.array.code);
// recopie dans le ArrayList
liste = new ArrayList<>();
for (int i=0; i<noms.length; ++i) {
    liste.add(new Etudiant(noms[i], code[i]));
}
```

Les données sont représentées dans un ArrayList volatile : quand on ferme l'activité, les données sont perdues. On peut définir une classe Application et mettre le tableau ainsi que son initialisation dedans. Ainsi, le tableau devient disponible dans toutes les activités

Affichage de la liste

L'affichage de la liste est fait par un RecyclerView: une vue qui intègre un défilement automatique et qui veille à économiser la mémoire pour l'affichage.

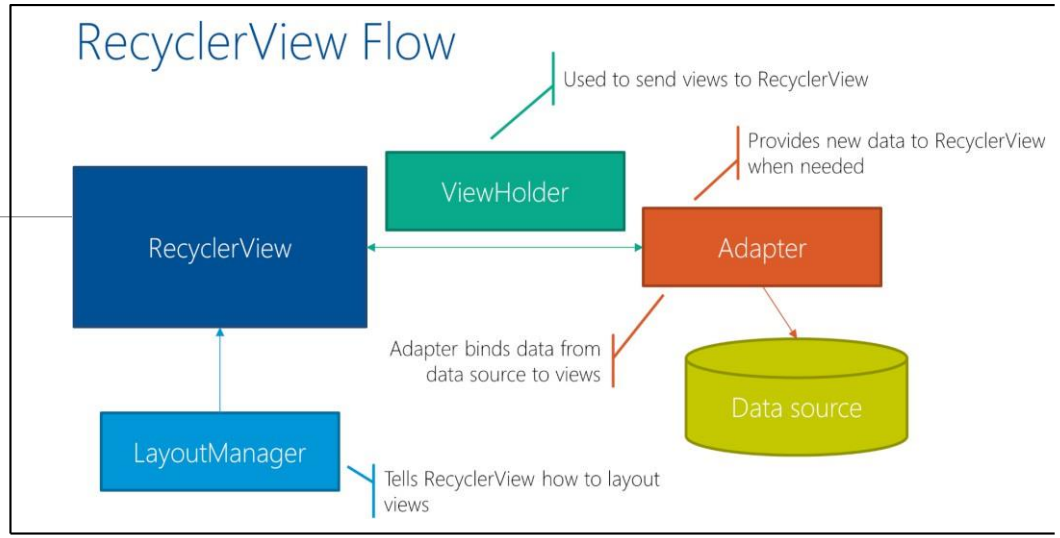
```
<androidx.recyclerview.widget.RecyclerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/recycler"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Mise en place du layout d'activité: En utilisant un ViewBinding :

```
private ArrayList<Etudiant> liste;
private ActivityMainBinding utlisiterf ;
@Override
protected void onCreate(Bundle savedInstanceState)
{
    // mettre en place le layout contenant le RecyclerView
    super.onCreate(savedInstanceState);
    utlisiterf = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(utlisiterf.getRoot());

    // initialisation de la liste avec les ressources
    liste = new ArrayList<>();
    ....
```

Relations entre la vue et les données



RecyclerView: C'est le composant qui vous permet de présenter une liste déroulable d'éléments. Il gère la présentation de la liste et le recyclage des éléments de la liste qui ne sont plus visibles à l'écran, d'où le nom "RecyclerView".

ViewHolder: C'est une classe interne statique dans l'Adapter qui contient les références aux vues dans un élément de liste. L'objectif du ViewHolder est de fournir ces vues à l'Adapter pour que ce dernier puisse les mettre à jour avec les données correspondantes. En recyclant les instances de ViewHolder et donc les vues qu'ils contiennent, le RecyclerView peut fonctionner de manière très efficace sans avoir besoin de créer de nouvelles vues pour chaque élément de données.

Adapter: Le rôle de l'Adapter est de faire le lien entre les données de la liste (la "Data source") et le RecyclerView lui-même. Il crée des ViewHolder et les relie aux données. Quand des données changent ou doivent être ajoutées, c'est l'Adapter qui s'en charge et met à jour le RecyclerView.

Data source: C'est l'ensemble des données que vous voulez afficher dans le RecyclerView. Cela peut être une liste, une base de données, ou tout autre type de structure de données contenant les éléments à présenter dans la liste.

LayoutManager: C'est un composant qui travaille avec le RecyclerView pour déterminer comment les éléments doivent être disposés à l'écran. Il existe différents types de LayoutManagers (par exemple, LinearLayoutManager, GridLayoutManager, StaggeredGridLayoutManager) qui vous permettent de présenter vos listes sous différentes formes, comme des listes verticales ou horizontales, des grilles, etc.

Recette pour les Liste

Étape A — Data (le modèle)

Une classe Etudiant + une ArrayList<Etudiant>

Étape B — Item UI

Un layout d'une ligne (ex: etudiant.xml)

Étape C — Adapter (avec ViewHolder)

3 méthodes seulement à retenir:

- getItemCount() : combien d'items
- onCreateViewHolder() : créer la "carte" (inflate)
- onBindViewHolder() : remplir la carte avec les données

Étape D — Brancher dans l'activité

Dans onCreate() :

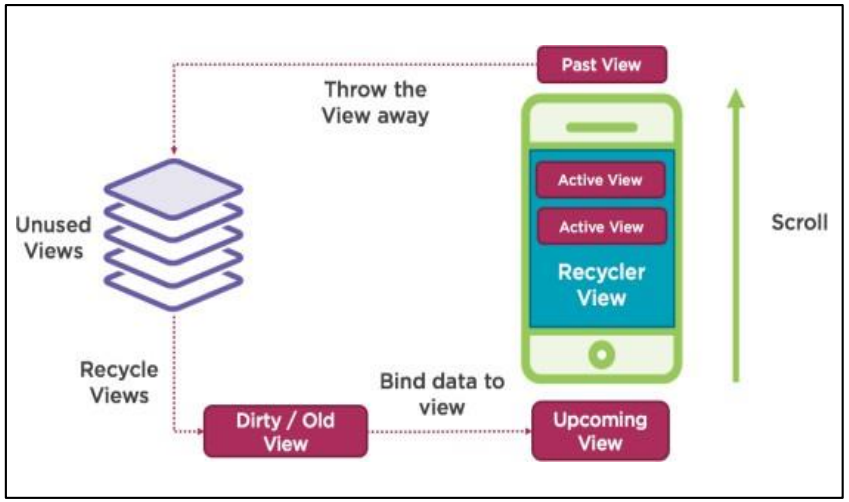
- recycler.setLayoutManager(new LinearLayoutManager(this))
- recycler.setAdapter(adapter) (+ optionnel setHasFixedSize(true) si tailles identiques)

Relations entre la vue et les données

La vue sert à afficher les éléments de la liste: un nombre d'éléments limité seront visibles en même temps (la hauteur de la liste et des éléments.)

Le principe du RecyclerView : ne gérer que les éléments visibles (les autres non mémorisés).

- Après défilement de la liste, Le RecyclerView demande alors à l'adaptateur de lui instancier (inflate) les vues pour afficher les éléments.
- Les vues qui deviennent invisible à cause du défilement vertical sont recyclées et renvoyées de l'autre côté mais en changeant seulement le contenu à afficher.
- Pour permettre ce recyclage, les vues associées à chaque élément doivent être soit recréées, soit réaffectées à l'aide des mini-containers : ViewHolders (une classe associée à une donnée de base, ex: un étudiant).



- Imagine 1000 étudiants.
- Sur l'écran, on voit seulement ~10 lignes.
- RecyclerView **ne crée pas 1000 vues**, il crée ~10–12 cartes et les **réutilise** quand on scroll.
- "Recyclage = même carte, données changent".

Exemple de ViewHolder

Implémenter un layout d'item, par exemple : res/layout/etudiant.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
    <RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"  >

        <TextView android:id="@+id/nom_prenom" />
        <TextView android:id="@+id/code"  />

    </RelativeLayout>
```

Ce layout sera instancié pour chaque « etudiant » visible dans le RecyclerView : On va utiliser son ViewBinding : la classe EtudiantBinding, pour accéder facilement aux vues.

La classe EtudiantViewHolder mémorise un EtudiantBinding: l'ensemble des vues représentant un étudiant à l'écran.
Ce ViewBinding sera fourni par l'adaptateur.
La méthode setEtudiant met à jour ces vues à partir de la donnée passée en paramètre. Cette méthode est appelée par l'adaptateur lors du recyclage.

```java
public class EtudiantViewHolder extends RecyclerView.ViewHolder
{
    private EtudiantBinding utinterf ;
    public EtudiantViewHolder(EtudiantBinding ui) {
        super(ui.getRoot());
        this.utinterf = utinterf;
    }
    public void setEtudiant(Etudiant etudiant) {
        utinterf.nom_prenom.setText(etudiant.getNom_prenom());
        utinterf.code_etud.setText(
                Integer.toString(etudiant.getCode_etud()));
    }
}
```

Recette pour les Liste

Étape A — Data (le modèle)

Une classe Etudiant + une ArrayList<Etudiant>

Étape B — Item UI

Un layout d'une ligne (ex: etudiant.xml)

Étape C — Adapter (avec ViewHolder)

3 méthodes seulement à retenir:

- getItemCount() : combien d'items
- onCreateViewHolder() : créer la "carte" (inflate)
- onBindViewHolder() : remplir la carte avec les données

Étape D — Brancher dans l'activité

Dans onCreate() :

- recycler.setLayoutManager(new LinearLayoutManager(this))
- recycler.setAdapter(adapter) (+ optionnel setHasFixedSize(true) si tailles identiques)

Rôle de l’adaptateur

L’adaptateur permet d’afficher un élément à tel endroit dans la liste : Il cherche les données et instancie/recycle un ViewHolder avec les valeurs. L’adaptateur est une classe qui :

- Stocke et gère les données : liste, connexion à une base de donnée, etc.
- Crée et remplit les vues d’affichage des items à la demande du RecyclerView.

Pour définir un adaptateur :

- Etendre la classe RecyclerView.Adapter (une classe générique) et lui indiquer la classe des ViewHolder.
- Définir le constructeur et Surcharger trois méthodes:
  - int getItemCount() : retourne le nombre d’éléments de la liste.
  - onCreateViewHolder(ViewGroup parent, int viewType) : permet de créer les ViewHolder.
  - onBindViewHolder (ViewHolder holder, int position): permet de recycler les ViewHolder.

Exemple d'adaptateur

```java
public class EtudiantAdapter extends RecyclerView.Adapter<EtudiantViewHolder>
{
    private final List<Etudiant> liste;
    EtudiantAdapter(List<Etudiant> liste) {
        this.liste = liste;
    }

    @Override
    public int getItemCount()
    { return liste.size();
    }

    @Override
    public EtudiantViewHolder
    onCreateViewHolder(ViewGroup parent, int viewType)
    {
        EtudiantBinding binding =
                EtudiantBinding.inflate( LayoutInflater.from(parent.getContext()),  parent,  b: false);
        return new EtudiantViewHolder(binding);
    }

    @Override
    public void onBindViewHolder(EtudiantViewHolder holder, int position)
    {
        Etudiant etudiant = liste.get(position);
        holder.setEtudiant(etudiant);
    }
```

Obtenez l'élément de votre jeu de données à cette position

Remplacez le contenu de la vue avec cet élément

Recette pour les Liste

Étape A — Data (le modèle)

Une classe Etudiant + une ArrayList<Etudiant>

Étape B — Item UI

Un layout d'une ligne (ex: etudiant.xml)

Étape C — Adapter (avec ViewHolder)

3 méthodes seulement à retenir:

- getItemCount() : combien d'items
- onCreateViewHolder() : créer la "carte" (inflate)
- onBindViewHolder() : remplir la carte avec les données

Étape D — Brancher dans l'activité

Dans onCreate() :

- recycler.setLayoutManager(new LinearLayoutManager(this))
- recycler.setAdapter(adapter) (+ optionnel setHasFixedSize(true) si tailles identiques)

RecyclerView: Configuration de l'affichage

Pour mettre en page des ViewHolders (en liste, en tableau, en grille, …) . . sachant que le RecyclerView est chargé de leur affichage et défilement:

- Etape 1: dans MainActivity, il est important d'indiquer au RecyclerView si les ViewHolder ont tous la même taille ou pas

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    //fixer la dimention à une constante
    utlisiterf.recycler.setHasFixedSize(true);
```

- Etape 2 (**Indispensable**) : Définir le LayoutManager : spécifie comment organiser les éléments : en liste, en tableau, en grille, …

```
// layout manager
RecyclerView.LayoutManager lm =  new LinearLayoutManager( context: this);
utlisiterf.recycler.setLayoutManager(lm);
```

```
<androidx.recyclerview.widget.RecyclerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/recycler"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutManager= "androidx.recyclerview.widget.LinearLayoutManager"/>
```

RecyclerView: Type LayoutManager

Les types des LayoutManagers:

- Un LinearLayoutManager organise les éléments en liste, en leur donnant tous la même taille.

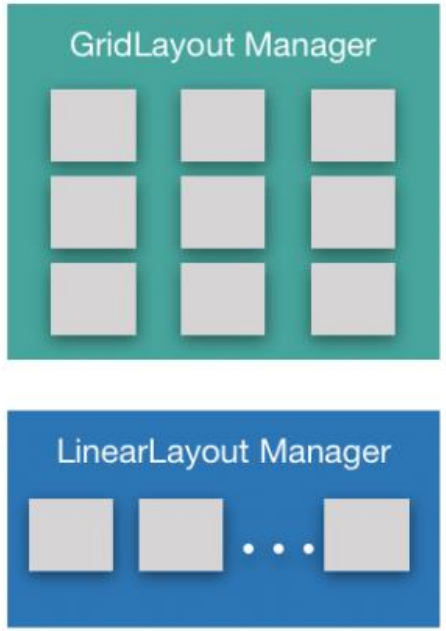
```
LinearLayoutManager lm =
        new LinearLayoutManager( context: this,
                RecyclerView.HORIZONTAL,     // direction : orientation du defilement
                reverseLayo
```

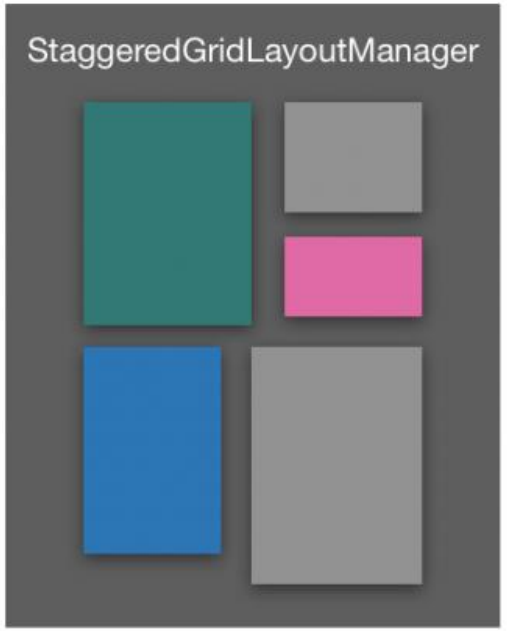
- GridLayoutManager arrai
  paramètre :

```
GridLayout
```

- StaggeredGridLayoutMan
  (ou largeurs, selon la direc

```
StaggeredGr
```





tutorial.eyehunts.com

RecyclerView: Séparateur entre items

Un RecyclerView n'affiche pas de ligne de séparation entre les élements. On peut l'ajouter en utilisant DividerItemDecoration dans la méthode en create de la main Activity:

```
// séparateur entre Items
DividerItemDecoration dividerItemDecoration =
        new DividerItemDecoration(
                context: this, DividerItemDecoration.VERTICAL);
utlisiterf.recycler.addItemDecoration(dividerItemDecoration);
```

Actions sur la liste

Deux types de modifications peuvent arriver à une liste:

- Modifications extérieures au RecyclerView : le programme Java modifie les données directement dans le ArrayList.
- Modifications effectuées par le RecyclerView : l'utilisateur va le déclencher suite à une action sur les éléments (clics, glissés. . .)

Toute modification extérieure sur la liste des éléments doit être signalée à l'adaptateur afin qu'à son tour il puisse prévenir le RecyclerView.

Selon la modification, il faut appeler :

- notifyItemChanged(int pos) : l'élément de cette position a été modifié
- notifyItemInserted(int pos) : un élément a été inséré à cette position
- notifyItemRemoved(int pos) : cet élément a été supprimé
- notifyDataSetChanged() pour aviser et actualiser les données de la liste (tri, rechargement. . .)

Actions sur la liste : Problème de Clic sur un élément

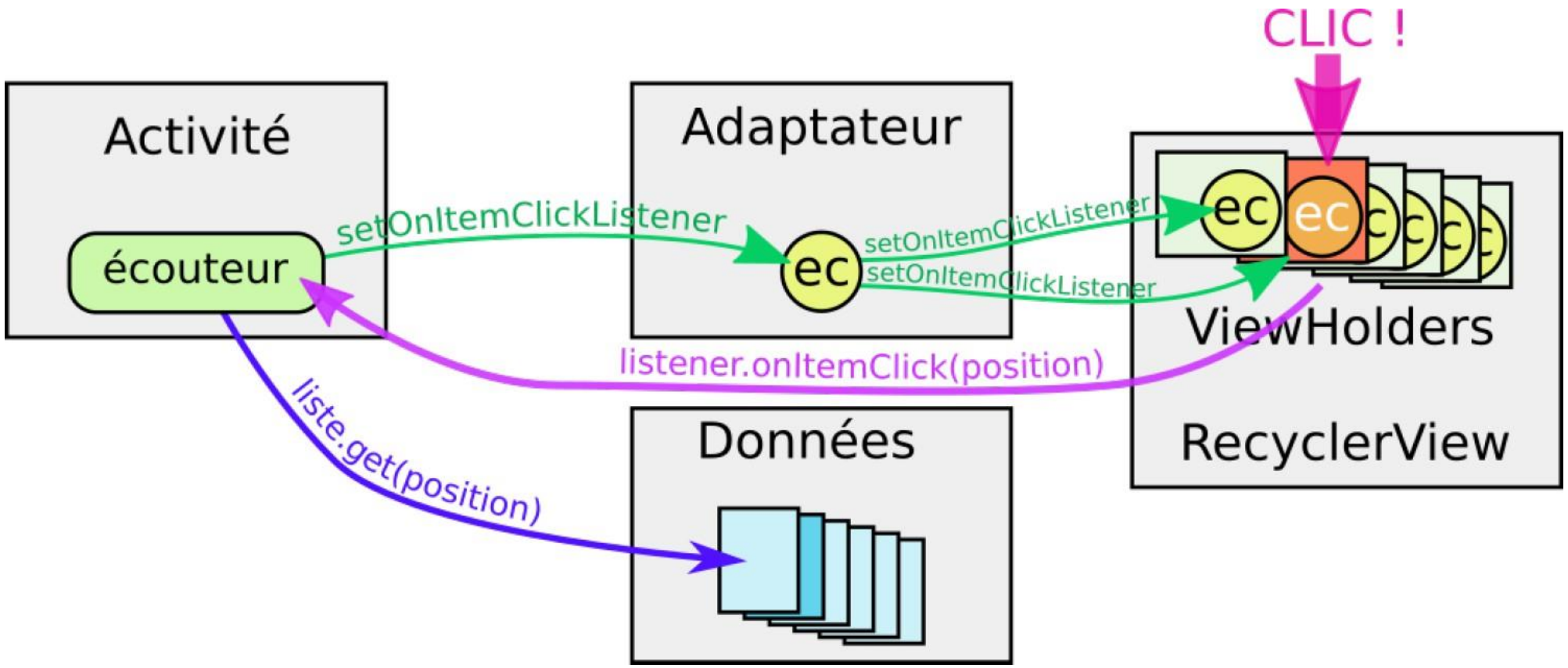
Dans Android, rien n'est prévu pour les clics sur les éléments. On doit construire une architecture d'écouteurs:

- L'activité MainActivity veut être prévenue quand l'utilisateur clique sur un élément de la liste.
- L'objet qui reçoit les événements utilisateur est le ViewHolder: implémenter la méthode onClick(View v) de l'interface View.OnClickListener comme un simple Button.
- Un RecyclerView regroupe plusieurs ViewHolder ; chacun peut être cliqué (un à la fois): il ne connaît pas l'activité.

→ faire le lien entre ces ViewHolder et l'activité : passer par l'adaptateur, le seul qui soit au contact des deux.

- Il faut que l'activité définisse un écouteur de clics et le fournisse à l'adaptateur.
- L'adaptateur transmet cet écouteur à tous les ViewHolder qu'il crée ou recycle.
- Chaque ViewHolder possède donc cet écouteur et peut le déclencher le cas échéant.

Actions sur la liste : Problème de Clic sur un élément



Actions sur la liste : Problème de Clic sur un élément

Au niveau l'activité :

L'adaptateur définit une interface que d'autres classes vont implémenter, par exemple une référence de méthode de l'activité.

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // adaptateur
        super.onCreate(savedInstanceState);
        EtudiantAdapter adapter = new EtudiantAdapter(liste);
        // écouteur pour les clics
        adapter.setOnItemClickListener(this::onItemClick);
    }
    private void onItemClick(int position) {
        Etudiant etudiant = liste.get(position);
        // utiliser etudiant  ...
    }
```

Actions sur la liste : Problème de Clic sur un élément

Au niveau de l'adaptateur:

Notre écouteur de clics: Ajouter une interface publique dans l'adaptateur :

```
public class EtudiantAdapter extends RecyclerView.Adapter<EtudiantViewHolder>
{
        public interface OnItemClickListener {
            void onItemClick(int position);
        }
        private OnItemClickListener listener;
        public void setOnItemClickListener(OnItemClickListener l) {
        this.listener = l;
    }
```

L'adaptateur fournisse cet écouteur aux ViewHolders :

```
public void onBindViewHolder(EtudiantViewHolder holder, ... )
{
    ....
    holder.setOnItemClickListener(this.listener);
}
```

Actions sur la liste : Problème de Clic sur un élément

Au niveau des ViewHolders:

Les ViewHolders doivent mémoriser cet écouteur : on fait référence à l'interface définie dans la classe EtudiantAdapter avec View.OnClickListener.

```
public class EtudiantViewHolder extends RecyclerView.ViewHolder implements View.OnClickListener{

        private final EtudiantBinding ui;
        private EtudiantAdapter.OnItemClickListener listener;

        public void setOnItemClickListener(EtudiantAdapter.OnItemClickListener l) {
        this.listener = l;
         }
```

Les ViewHolders doivent aussi recevoir les événements puis déclencher l'écouteur :La méthode getAdapterPosition() retourne la position de ce ViewHolder dans son adaptateur.

```
public EtudiantViewHolder(@NonNull EtudiantBinding ui) {
        super(ui.getRoot());
        this.ui = ui;
        itemView.setOnClickListener(this);
    }
    @Override public void onClick(View v) {
        listener.onItemClick(getAdapterPosition());
    }
```